

JAVA_00 開發環境與使用工具

目錄

學習目標

完成本課程後，你將能夠獨立安裝和設定Java開發環境，並開始你的Java程式設計之旅！

1.

Java 可做什麼？

了解Java的應用領域和學習Java的價值



學習內容：

- ▶ Java應用領域 (Web、Android、企業系統)
- ▶ Java職涯發展方向

2.

Java 開發工具

認識Java開發所需的各種工具和軟體



學習內容：

- ▶ 整合開發環境 (IDE) 介紹
- ▶ 建置工具 (Maven vs Gradle)
- ▶ 測試與除錯工具
- ▶ JDK vs JRE 差異說明
- ▶ 版本控制工具 (Git)

3.

安裝 JDK

下載、安裝，並設定環境變數



學習內容：

- ▶ 下載 JDK
- ▶ 安裝驗證測試

4.

安裝 Eclipse

安裝Eclipse IDE並進行基本設定



學習內容：

- ▶ Eclipse版本下載
- ▶ 工作空間 (Workspace) 設定
- ▶ 基本介面介紹

1.Java 可做什麼？

探索Java在日常生活中的廣泛應用

 思考一下

你覺得你每天用的哪些東西，可能是 Java 做出來的？



銀行系統

很多銀行後台系統使用 Java EE，安全性高、穩定性強

☀ 生活例子：

ATM

自動提款機

線上網銀

⚡ 為什麼用Java？

金融系統需要極高的安全性和穩定性，Java的強型別檢查和成熟的安全機制正好符合需求。



電商網站

Java + Spring 常被用於大型電商後台

☀ 生活例子：

Momo

PChome

蝦皮 (部分功能)

⚡ 為什麼用Java？

電商網站需要處理大量交易和用戶請求，Java的高並發處理能力和豐富的框架生態系統是絕佳選擇。



手機應用程式 (Android)

Android 開發初期與核心支援語言就是 Java

☀ 生活例子：

LINE

Google 地圖

YouTube App

⚡ 為什麼用Java？

Android系統本身就是基於Java虛擬機器設計，雖然現在也支援Kotlin，但Java仍是Android開發的重要語言。



學校系統

很多學校 MIS 系統以 Java 開發

☀ 生活例子：

選課系統

教務系統

ePortfolio

⚡ 為什麼用Java？

學校系統需要長期穩定運作，Java的跨平台特性和企業級框架支援讓系統維護更容易。



遊戲平台

完整由 Java 編寫，跨平台執行

☀ 生活例子：

Minecraft (PC版)

⚡ 為什麼用Java？

Minecraft選擇Java是因為它的跨平台特性，讓遊戲可以在Windows、Mac、Linux上無縫運行。



交通票務系統

Java 被大量應用於自動售票與後台資料處理系統

☀ 生活例子：

高鐵訂票

自動售票機

⚡ 為什麼用Java？

交通系統需要24小時穩定運作，Java的可靠性和強大的多執行緒處理能力是關鍵因素。



企業後台管理系統

Java 是企業系統主力技術之一

☀ 生活例子：

ERP (企業資源管理)

CRM (客戶關係管理)

⚡ 為什麼用Java？

企業系統需要處理複雜的商業邏輯和大量數據，Java的物件導向設計和豐富的企業級框架正好滿足需求。



智慧裝置與機器人

Java 在嵌入式系統與遠端管理平台也被使用

☀ 生活例子：

工廠自動化控制

機器人後台

⚡ 為什麼用Java？

Java的「一次編寫，到處執行」特性讓它很適合在各種硬體平台上部署，簡化了開發和維護成本。

總結

從你每天用的手機App、網路購物、學校系統，到銀行ATM，Java都在幕後默默支撐著這些服務。
這就是為什麼學習Java如此有價值 - 它不只是一門程式語言，更是現代數位生活的重要基石！

2.Java 開發工具

Java開發環境完整指南

 思考一下

作為Java開發者，每天都會用到各種工具，
你知道哪些工具能讓你的開發效率倍增嗎？

(1) 整合開發環境 (IDE) : 程式開發的核心工作平台

(2) Java 開發套件 (JDK) : 基礎執行環境

(3) 建置管理工具 : 專案管理與相依性處理

(4) 其他重要工具 : 版本控制、測試、除錯工具

整合開發環境 (IDE)

| 什麼是IDE？

IDE就像是程式設計師的「數位工作室」，提供寫程式所需的所有工具。



IntelliJ IDEA

- 智慧程式碼提示
- 強大重構功能
- 業界廣泛使用



Eclipse

- 完全免費開源
- 豐富外掛生態
- 歷史悠久穩定



Visual Studio Code

- 輕量快速
- 跨平台支援
- 現代化介面



NetBeans

- Oracle官方支援
- 初學者友善
- 內建伺服器支援

Java 開發套件 (JDK)

| JDK 是什麼？

Java Development Kit - 包含編譯器、執行環境、除錯工具等核心組件。

主要版本選擇：

- **Oracle JDK** - Oracle官方商業版本
- **OpenJDK** - 開源免費版本，功能完整 - OpenLogic OpenJDK
- **Amazon Corretto** - AWS提供的免費版本
- **Azul Zulu** - Azul Systems的企業版本



初學者建議

推薦使用 OpenJDK 17 (LTS版本)

LTS = Long Term Support (長期支援版本)

建置管理工具

| 為什麼需要建置工具？

管理專案相依性、自動化編譯、打包、測試等流程。



Maven

特色：

- XML設定檔 (pom.xml)
- 標準專案結構
- 豐富的外掛生態
- 業界標準



Gradle

特色：

- Groovy/Kotlin DSL
- 更靈活的設定
- 增量建置
- Android官方工具

版本控制工具

| Git - 現代開發必備

追蹤程式碼變更、協作開發、備份程式碼的最佳工具。

Git 基本概念：

- Repository - 程式碼倉庫
- Commit - 提交變更記錄
- Branch - 分支開發
- Merge - 合併分支

熱門Git平台：

GitHub

全球最大程式碼託管平台

GitLab

支援私有部署的完整DevOps平台

測試與除錯工具

測試工具

JUnit

- Java單元測試的標準框架
- 簡單易用，支援各種測試場景

Mockito

- 模擬物件測試工具
- 測試複雜相依性的好幫手

除錯與效能工具

- **IDE內建除錯器** - 設中斷點、逐步執行
- **JProfiler** - 專業效能分析工具
- **VisualVM** - 免費的JVM監控工具
- **JConsole** - JDK內建的監控工具

3. 安裝 JDK

Java Development Kit - Java開發的基礎環境

什麼是 JDK ?

JDK (Java Development Kit) 是Java開發工具包，包含了開發Java應用程式所需的所有工具和資源。它包括Java編譯器 (javac)、Java虛擬機器 (JVM)、標準類別庫、以及其他開發工具。

步驟1:下載安裝檔

前往Oracle官網或Adoptium網站，下載對應的JDK安裝檔 (.msi 或 .exe)

步驟2:執行安裝程式

雙擊安裝檔，按照安裝精靈的指示完成安裝。建議使用預設安裝路徑。

步驟3:查看環境變數

查看PATH環境變數

步驟4:驗證安裝

開啟命令提示字元，輸入指令驗證安裝



<https://www.openlogic.com/openjdk-downloads>

⚠ 版本選擇建議

建議選擇 LTS (Long Term Support) 版本，Java 17 或更高版本。
這些版本具有長期支援，適合生產環境使用。

第一步驟：下載安裝檔

Java Version	Operating System	Architecture	Java Package	
17 ▼	Windows ▼	- Any - ▼	- Any - ▼	Reset

下載 JDK 時，請根據：

- 作業系統 (Windows / macOS / Linux)
- 電腦架構 (x64 / ARM / M1 / M2)

👉 簡單來說，就是「看你的電腦環境」來做選擇。

JAVA VERSION	OPERATING SYSTEM	ARCHITECTURE	JAVA PACKAGE	DOWNLOAD
17.0.16+8	Windows	x86 64-bit	JDK	.msi
17.0.16+8	Windows	x86 64-bit	JDK	.zip



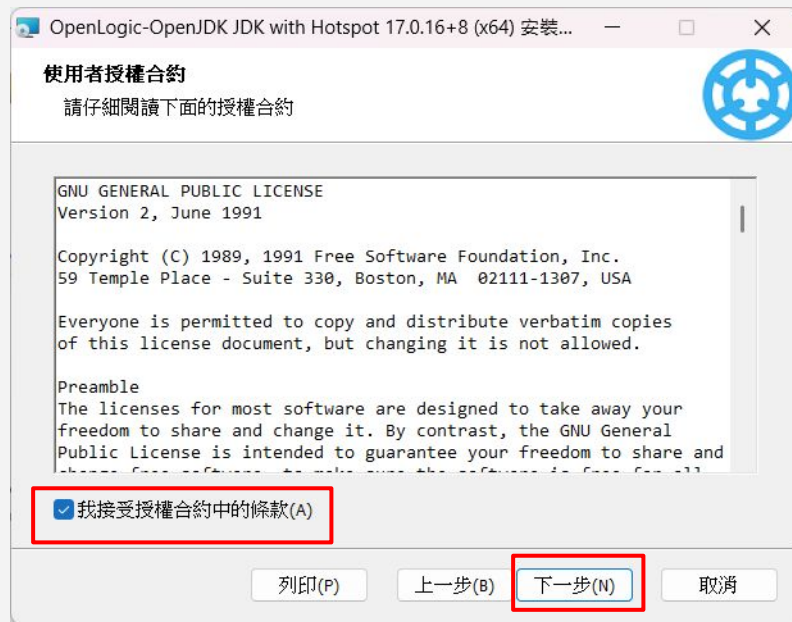
openlogic-openjdk-17.0.16+8-windows-x64.msi

第二步驟：執行安裝程式

1

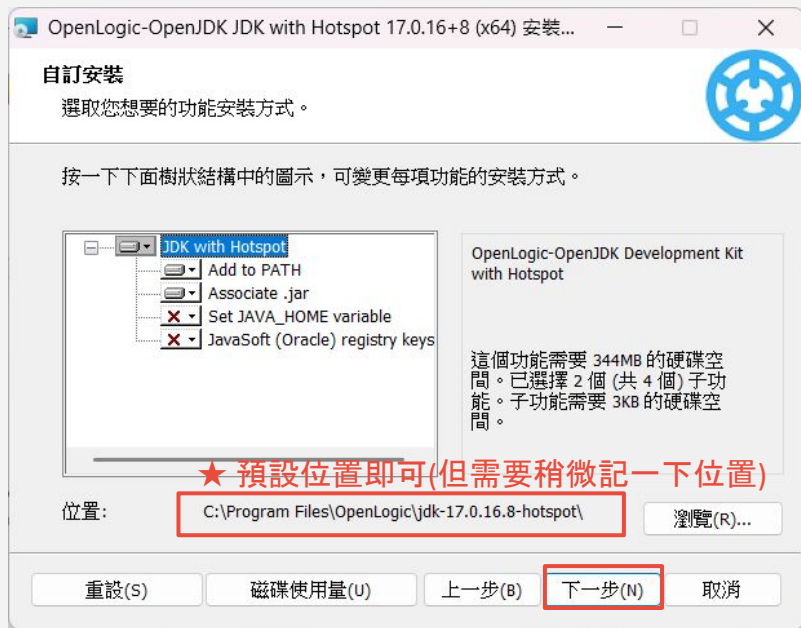


2



第二步驟：執行安裝程式

3



4



第二步驟：執行安裝程式

5



第三步驟：查看環境變數

1

設定 > 系統



2

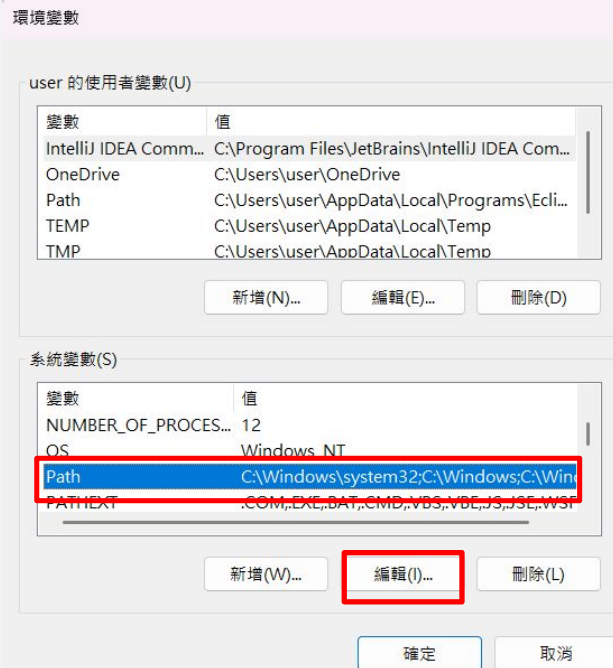
進階系統設定



第三步驟：查看環境變數



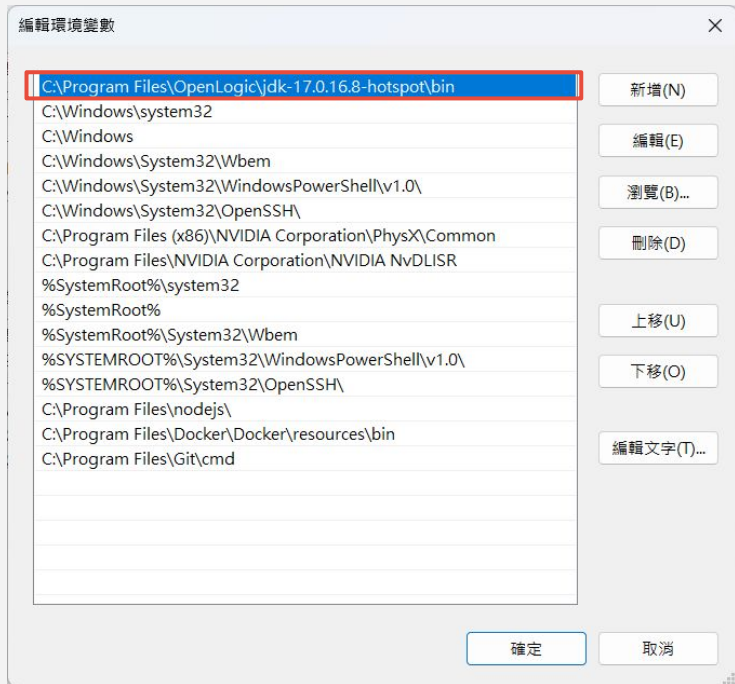
4 系統變數 > 找到「Path」> 編輯



第三步驟：查看環境變數

5

查看JDK是否存在



操作提示

★如果Path中同時存在多個JDK：

可以利用「上移」和「下移」來決定優先順序



- 提高搜尋優先級
- 更早被系統找到
- 優先使用此版本



- 降低搜尋優先級
- 較晚被系統找到
- 作為備用選項

第四步驟：驗證安裝

- 1 win + R打開「執行」視窗，並輸入「**cmd**」



- 2 輸入「**java -version**」，檢查是否設定成功

★確認與自己下載的版本是否相同

```
C:\Users\user>java -version
openjdk version "17.0.16" 2025-07-15
OpenJDK Runtime Environment Temurin-17.0.16+8 (build 17.0.16+8)
OpenJDK 64-Bit Server VM Temurin-17.0.16+8 (build 17.0.16+8, mixed mode, sharing)
```

- 非必要
- 輸入「**where java**」，可知道是哪一個檔案的JDK

★可以看到您的DK在哪個位置

```
C:\Users\user>where java
C:\Program Files\OpenLogic\jdk-17.0.16.8-hotspot\bin\java.exe
```

4. 安裝 Eclipse

Java整合開發環境

什麼是 Eclipse ?

Eclipse是一個免費、開源的整合開發環境(IDE)，主要用於Java開發。它提供了豐富的功能，包括程式碼編輯、偵錯、專案管理、版本控制等，是Java開發者最受歡迎的開發工具之一。

步驟1:下載安裝檔

前往 Eclipse 官方網站，下載對應的 IDE 安裝檔

步驟2:執行安裝程式

執行下載的安裝檔，按照安裝精靈的指示完成安裝

步驟3:設定工作區

首次啟動 Eclipse，選擇工作區 (Workspace) 位置

步驟4:驗證安裝

建立測試專案，確認 Eclipse 和 Java 環境運作正常

 前往下載 

<https://www.eclipse.org/downloads/packages/>

第一步驟：下載安裝檔

前往 Eclipse 官方網站，下載對應的 IDE 安裝檔

Eclipse IDE 2025-06 R Packages

Eclipse IDE for Enterprise Java and Web Developers

553 MB 279,854 DOWNLOADS



Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces, Yaml, Markdown, Web Services, JPA and Data Tools, Maven and Gradle, Git, and more.

Click [here](#) to raise an issue with the Eclipse Web Tools Platform. Maintainers will move opened issues to the right place.

Click [here](#) to raise an issue with the Eclipse Platform.

Click [here](#) to raise an issue with Maven integration for web projects.

Click [here](#) to raise an issue with Eclipse Wild Web Developer (incubating).



Windows | [x86_64](#) | [AArch64](#)
macOS [x86_64](#) | [AArch64](#)
Linux [x86_64](#) | [AArch64](#) | [riscv64](#)

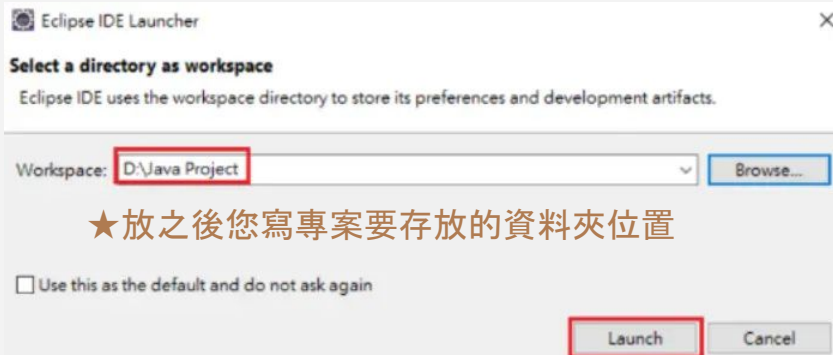
- 選擇 **Eclipse IDE for Java Developers**
- 下載適合作業系統的版本 (.exe 或 .zip)

第二步驟：執行安裝程式

1 選擇 Workspace（工作區） 的位置

📁 工作區是什麼？

- Eclipse 用來儲存你的**專案檔案**的資料夾
- 所有 Java 專案、原始碼、設定檔都會放在這裡



2 防毒軟體的安全性檢查 (非每個都會遇到)

Exclude Eclipse IDE from being scanned(從 Defender 的即時掃描中排除。)

✅ 優點: 啟動和編譯速度會明顯快很多。

⚠️ 缺點: 如果 Eclipse 裡有惡意檔案, 不會被掃描到。

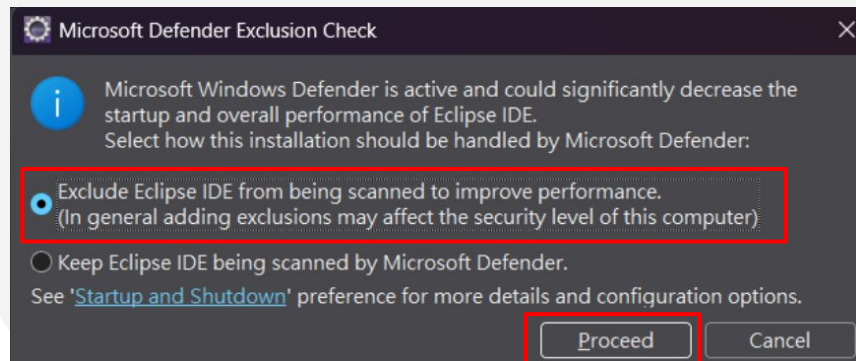
適合: 電腦平時安全、且只是開發用。

Keep Eclipse IDE being scanned(保持 Defender 掃描 Eclipse。)

✅ 優點: 安全性最高。

⚠️ 缺點: 啟動和編譯可能會變慢, 有時甚至造成 Eclipse 卡頓或啟動錯誤。

適合: 對安全要求特別高的人。



第三步驟：設定工作區

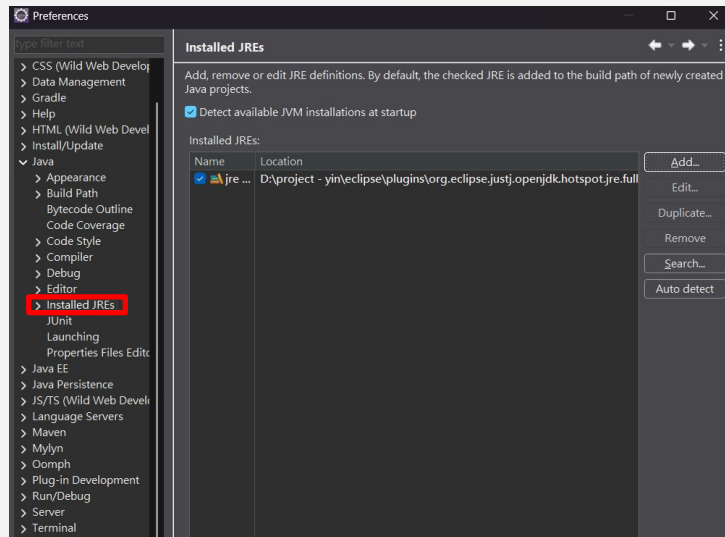
1 打開eclipse.exe

- > 下載後，解壓縮
- > 將檔案移置C槽或您常用的地方

名稱	修改日期	類型	大小
configuration	2025/9/8 下午 02:33	檔案資料夾	
dropins	2025/6/5 下午 02:33	檔案資料夾	
features	2025/9/3 上午 11:59	檔案資料夾	
p2	2025/9/8 下午 02:34	檔案資料夾	
plugins	2025/9/3 上午 11:59	檔案資料夾	
readme	2025/8/22 下午 01:12	檔案資料夾	
.eclipseproduct	2025/8/22 下午 01:11	ECLIPSEPRODUC...	1 KB
artifacts.xml	2025/9/3 上午 11:59	Microsoft Edge H...	1,052 KB
eclipse.exe	2025/8/22 下午 01:11	應用程式	546 KB
eclipse.ini	2025/8/22 下午 01:11	組態設定	2 KB
eclipse.exe	2025/8/22 下午 01:11	應用程式	258 KB

2 設定Installed JREs

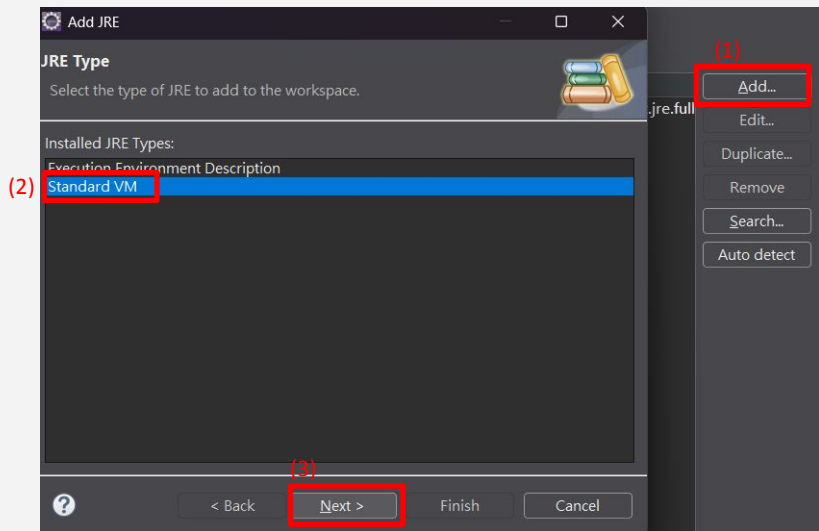
- > 位置: Window > Preferences
- > 左側「Java」>「Installed JREs」



第三步驟：設定工作區

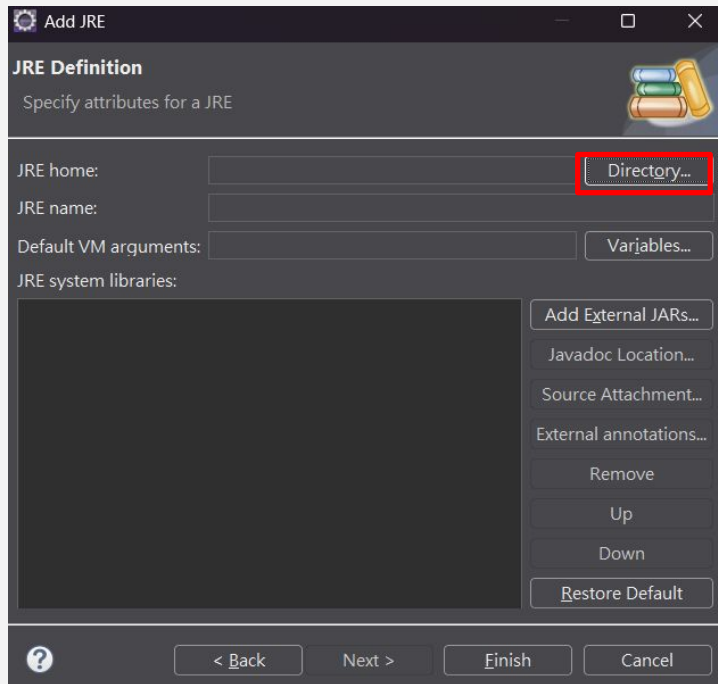
3

設定Installed JREs



4

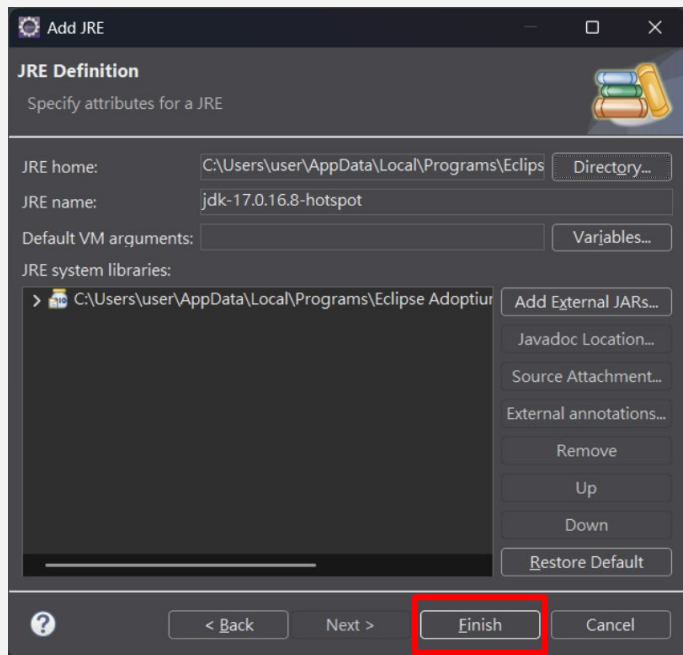
設定Installed JREs



第三步驟：設定工作區

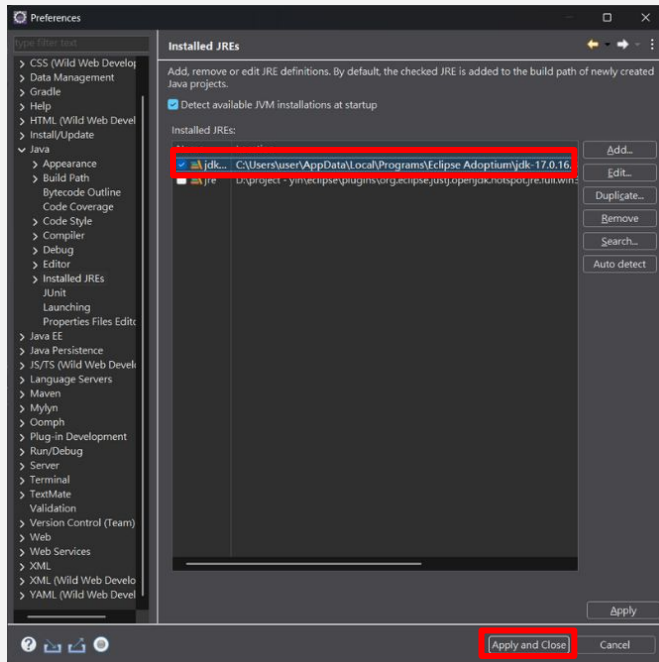
5

設定Installed JREs



6

設定Installed JREs -設定完成

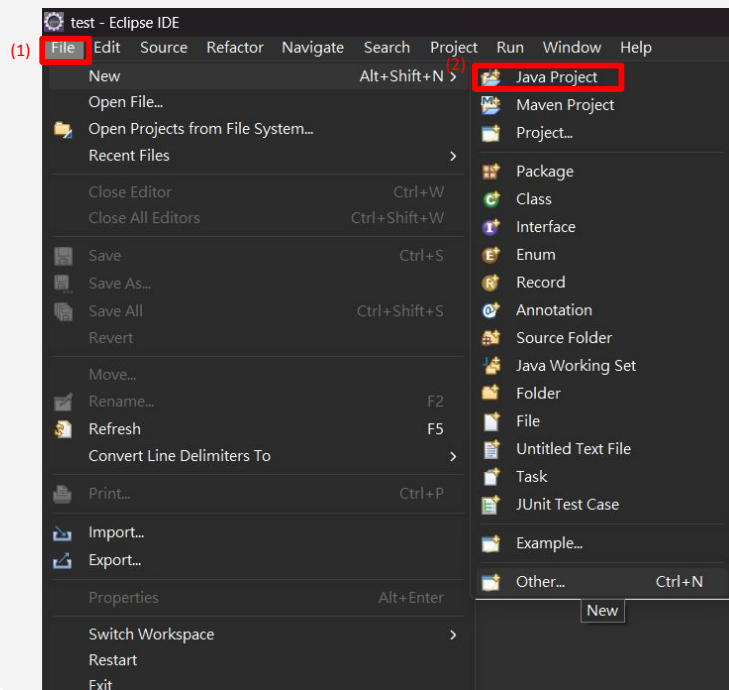


若專案使用 Grandle, 請參考：[專案使用 Grandle如何設定 JDK?](#)

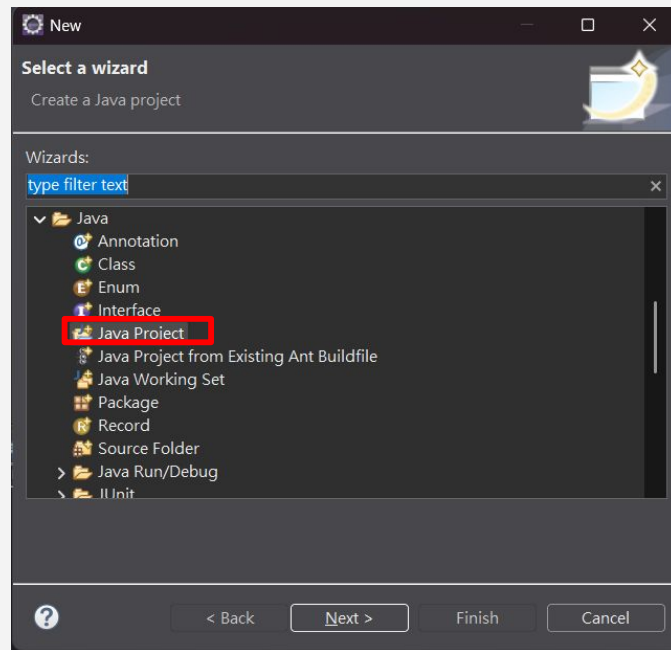
第四步驟：驗證安裝

1

新增第一支JAVA



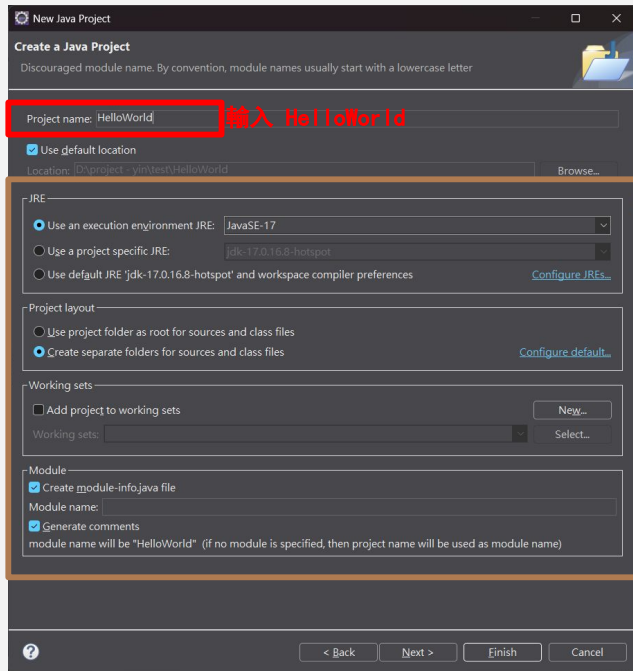
如果未看到Java Project
>請點選Other > Java > Java Project



第四步驟：驗證安裝

2

輸入專案名稱 **HelloWorld**,
並點擊完成

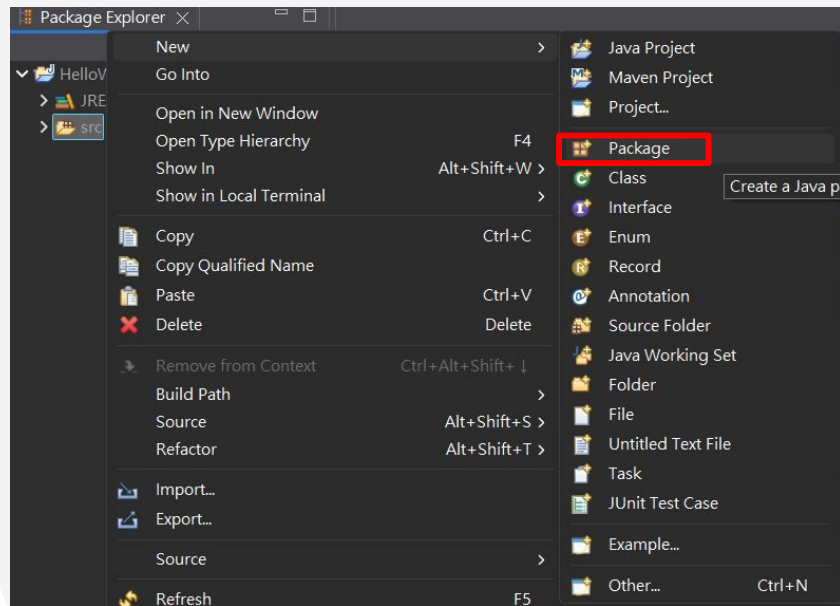


預設即可

3

建立一個 package

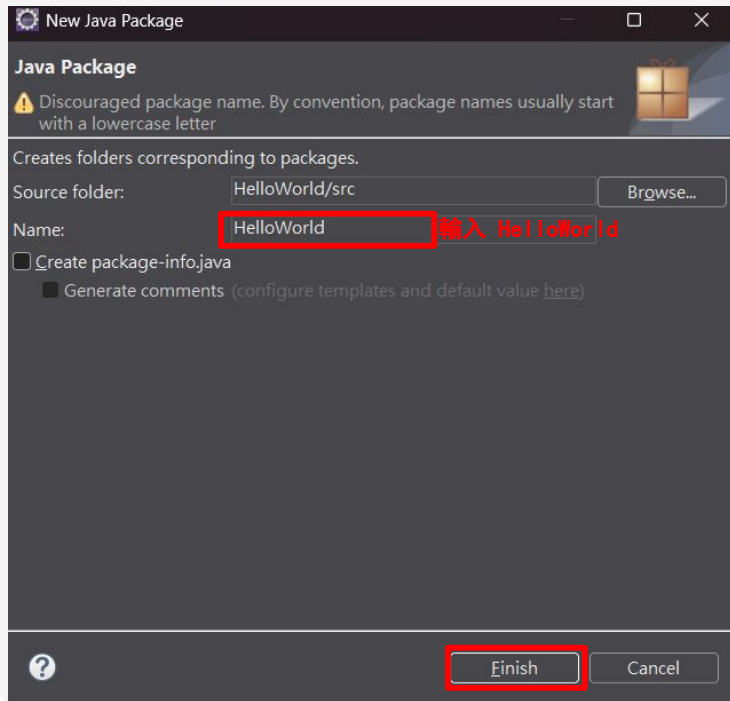
在 **Package Explorer** 左側找到 **src** → 右鍵 → New → Package



第四步驟：驗證安裝

4

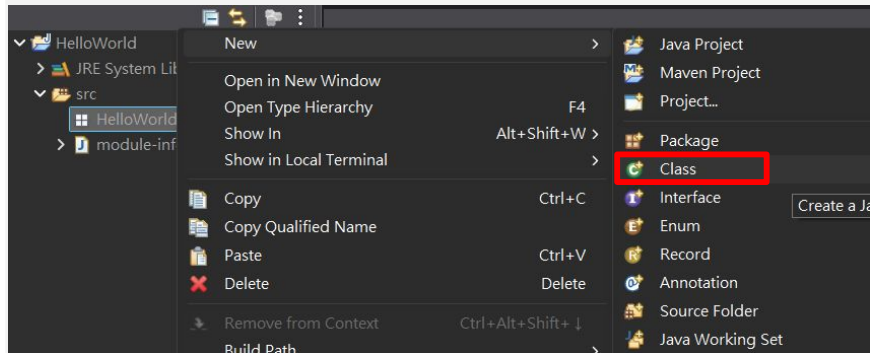
命名package為「HelloWorld」



5

在package下，建立一個 class

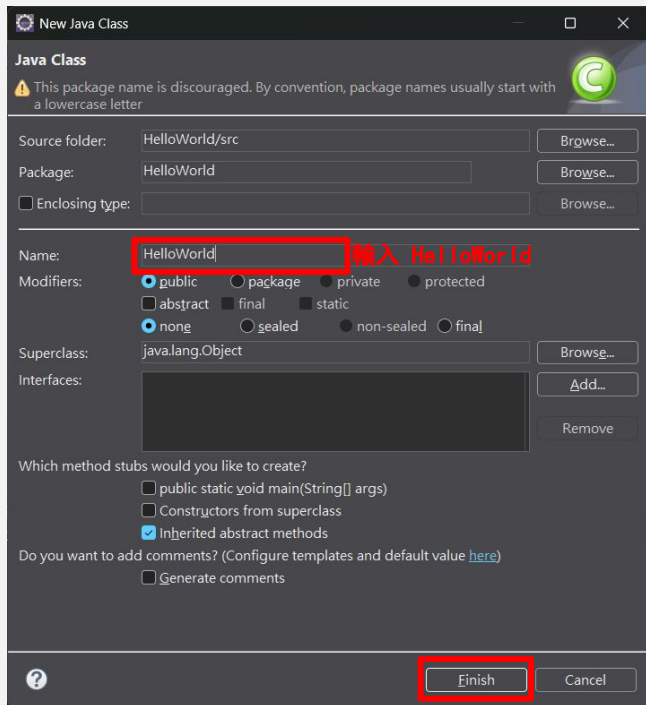
HelloWorld → 右鍵 → New → Class



第四步驟：驗證安裝

4

命名class Name為「HelloWorld」



New Java Class

Java Class

⚠ This package name is discouraged. By convention, package names usually start with a lowercase letter

Source folder: HelloWorld/src Browse...

Package: HelloWorld Browse...

☐ Enclosing type: Browse...

Name: HelloWorld 輸入 HelloWorld

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: java.lang.Object Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?

☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

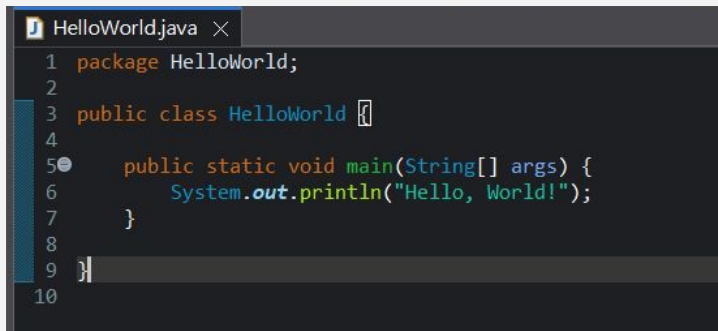
Finish Cancel

5

撰寫程式碼

請輸入此段程式碼：

```
public static void main(String[] args) {  
    System.out.println("Hello, World!");  
}
```

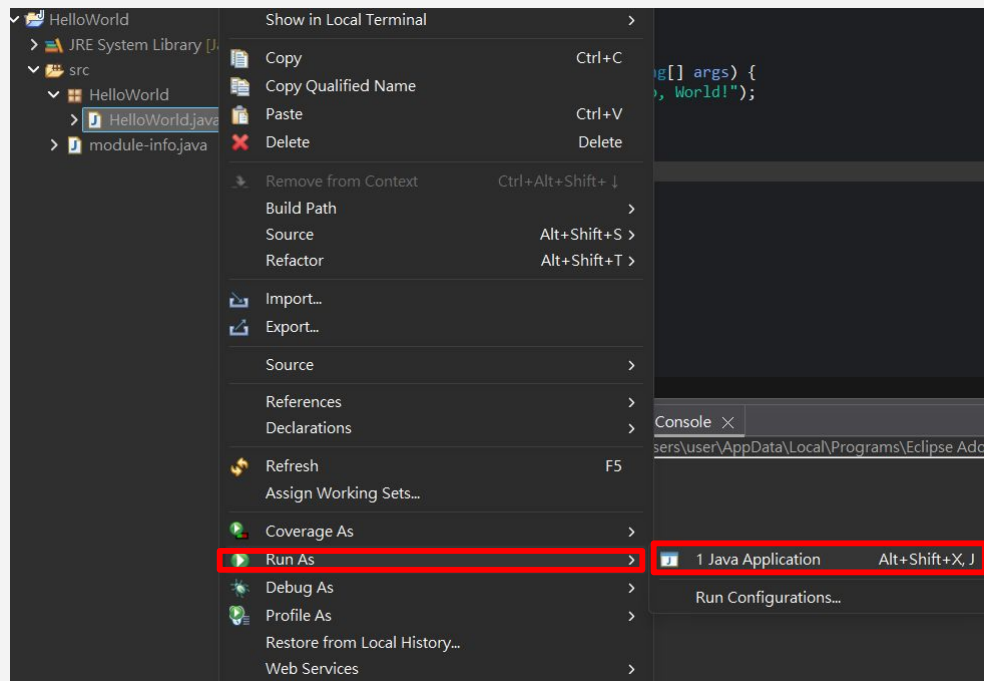


```
1 package HelloWorld;  
2  
3 public class HelloWorld {  
4  
5     public static void main(String[] args) {  
6         System.out.println("Hello, World!");  
7     }  
8  
9 }  
10
```

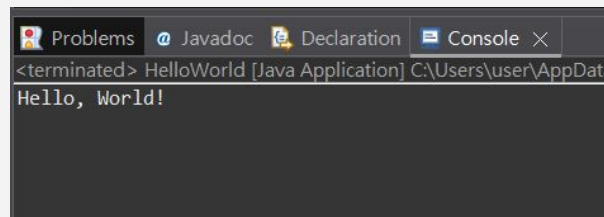
第四步驟：驗證安裝

6

輸出



> HelloWorld.java → 右鍵 → Run As → Java Application
> 輸出結果：

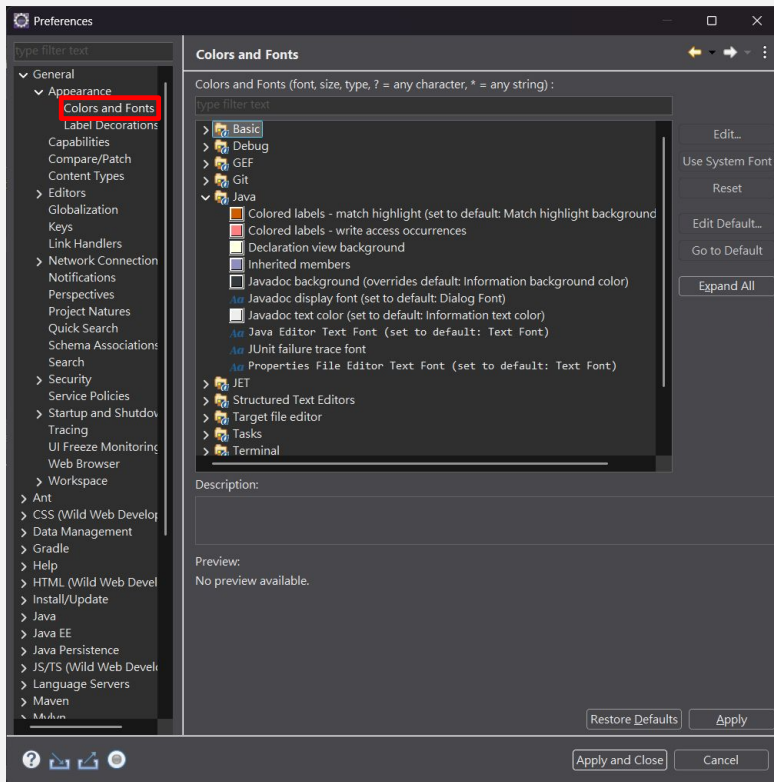




Eclipse 基礎設定



字型與外觀



Window

→ Preferences

→ General

→ Appearance

→ Colors and Fonts

→ 設定好後點選 Apply 或 Apply and Close



END

課外補充

課外補充目錄

1. 環境變數命名規則

2. 環境變數是甚麼

3. 環境變數四大用途

4. 編輯Path變數

5. 環境變數 V.S Path

6. eclipse錯誤

環境變數命名規則

標準命名

```
JAVA_HOME      - 主要版本  
JAVA_HOME_11   - Java 11  
JAVA_HOME_17   - Java 17  
JAVA_HOME_21   - Java 21
```

使用場景：

- Maven、Gradle 工具
- IDE 開發環境
- 團隊協作項目

自訂命名

```
MY_JAVA_PATH  
PROJECT_JAVA_HOME  
CUSTOM_JDK_PATH  
DEV_JAVA_17
```

適用情況：

- 個人腳本使用
- 特殊項目需求
- 多環境管理



重要提醒：

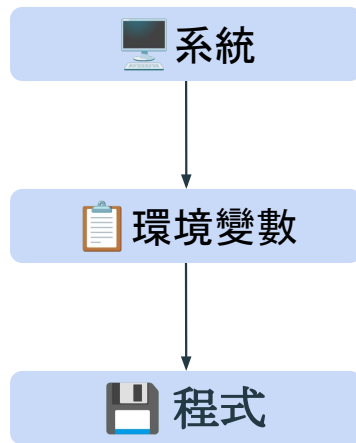
- 前綴統一使用 **JAVA_HOME** 確保工具相容性
- 多版本管理可在後面加版本號：_11、_17、_21
- 變數名稱建議使用 **全大寫字母**，用 **底線分隔**

什麼是環境變數



環境變數就像是「**程式的記事本**」

記錄著程式運行時需要的重要資訊



環境變數四大用途



配置管理

不同環境使用不同設定



系統路徑

告訴電腦去哪找程式



安全保護

敏感資訊不寫在程式碼



跨平台

讓程式在不同系統運行

環境變數四大用途 - 配置管理

✗ 沒有環境變數

每次換環境都要改程式碼

✓ 有環境變數

同一個程式, 不同環境自動適應

環境變數四大用途 - 系統路徑

📌 PATH 環境變數就像是「通訊簿」，告訴系統去哪些地方找程式



系統尋找程式的過程：

1. 你在命令提示字元輸入：`python`
2. 系統問：「`python.exe` 在哪裡？」
3. 系統查看 PATH 環境變數中的目錄清單
4. 依序到每個目錄找 `python.exe`

PATH 就是一串目錄路徑，用分號隔開：

📁 C:\Python39 (先找這裡)
📁 C:\Windows\System32 (再找這裡)
📁 C:\Program Files\Git\bin (最後找這裡)

```
PATH=C:\Python39;C:\Windows\System32;C:\Program Files\Git\bin
```

這樣你就可以在任何地方直接執行程式！

環境變數四大用途 - 系統保護

敏感資訊**絕對不要**寫在程式碼裡！

✗ 危險做法

```
password = "123456"  
api_key = "secret_key_xyz"
```

✓ 安全做法

```
password =  
os.getenv("DB_PASSWORD")  
api_key =  
os.getenv("API_KEY")
```

環境變數 - 總結

環境變數讓你的程式：

- ✓ 更靈活 - 適應不同環境
- ✓ 更安全 - 保護敏感資訊
- ✓ 更好維護 - 不用改程式碼
- ✓ 更專業 - 符合業界標準

編輯Path變數

Path 變數設定

```
%JAVA_HOME_17%\bin
```

- ▶ **%JAVA_HOME_17%** - 環境變數引用
- ▶ **\bin** - Java 執行檔路徑
- ▶ **_17** - 版本識別標記 (自訂)



作用說明

讓系統能在任何位置執行 Java 命令
(java、javac、jar 等)

版本管理優勢

- 1 多版本共存**
同時安裝 Java 11、17、21
- 2 快速切換**
修改 Path 變數即可切換版本
- 3 腳本自動化**
批次檔案可動態設定版本
- 4 專案隔離**
不同專案使用不同 Java 版本

1. 系統查找
尋找 %JAVA_HOME_17%



2. 路徑展開
替換為實際路徑



3. 執行命令
找到並執行 java.exe

新增環境變數

主要用途

- 定義 Java 安裝路徑
- 供其他程式引用
- 版本管理與切換
- 腳本自動化使用

特點

- 建立變數儲存路徑
- 不直接影響命令執行
- 需要被其他設定引用
- 便於統一管理路徑

VS

修改 Path 變數

主要用途

- 讓系統找到執行檔
- 任意位置執行命令
- 命令列工具使用
- IDE 整合開發

特點

- 直接影響命令執行
- 系統搜尋執行檔路徑
- 可引用環境變數
- 立即生效使用



關鍵差異總結

新增環境變數 = 定義路徑儲存位置（變數宣告）

修改 Path = 告訴系統去哪裡找執行檔（實際使用）

兩者需要**搭配使用**才能完整設定 Java 開發環境！

Eclipse 錯誤



關鍵錯誤訊息：

Could not resolve all dependencies for configuration ':compileClasspath'.

> Failed to calculate the value of task ':compileJava' property 'javaCompiler'.

> Cannot find a Java installation on your machine (Windows 11 18.0) → 系統找不到 Java 安裝
matching: [languageVersion=17, vendor=any vendor, implementation=vendor-specific]

↳ 找不到 Java 17

```
[Gradle Operations]
> Learn more about toolchain repositories at https://docs.gradle.org/8.14.3/userguide/toolchains.html#sub:download\_repositories.
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

CONFIGURE FAILED in 729ms

FAILURE: Build failed with an exception.

* What went wrong:
Could not resolve all dependencies for configuration ':compileClasspath'.
> Failed to calculate the value of task ':compileJava' property 'javaCompiler'.
  > Cannot find a Java installation on your machine (Windows 11 10.0 amd64) matching: {languageVersion=17, vendor=any vendor, implementation=vendor-specific, nativeImage=any}

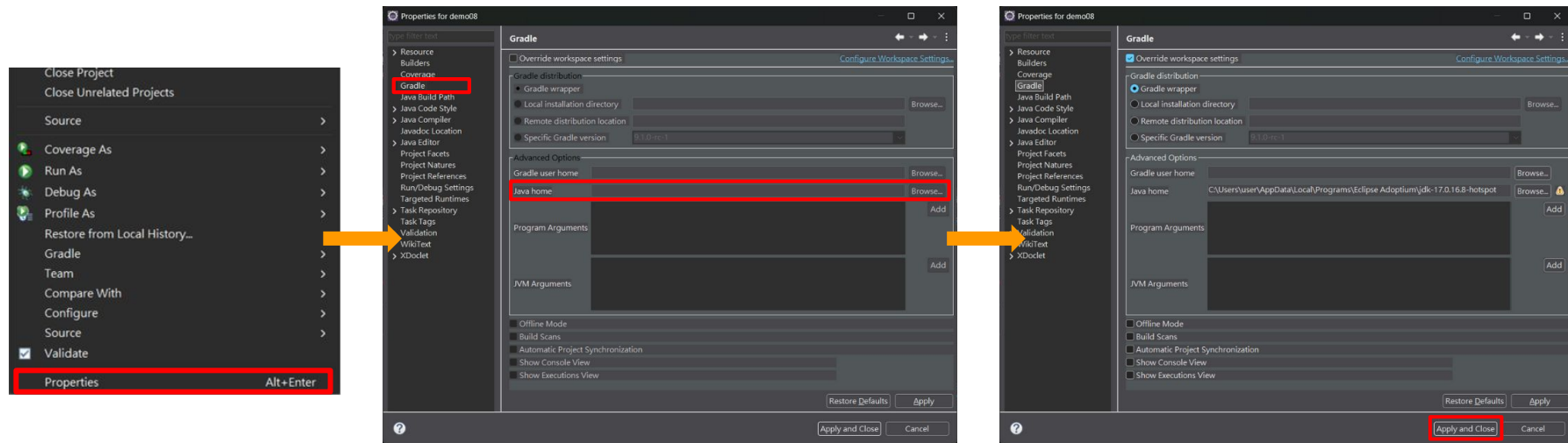
* Try:
> Learn more about toolchain auto-detection and auto-provisioning at https://docs.gradle.org/8.14.3/userguide/toolchains.html#sec:auto\_detection.
> Learn more about toolchain repositories at https://docs.gradle.org/8.14.3/userguide/toolchains.html#sub:download\_repositories.
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights.
> Get more help at https://help.gradle.org.

CONFIGURE FAILED in 663ms
> Task :nothing UP-TO-DATE
```

Eclipse 錯誤

解決方案: 手動指定 Java Home

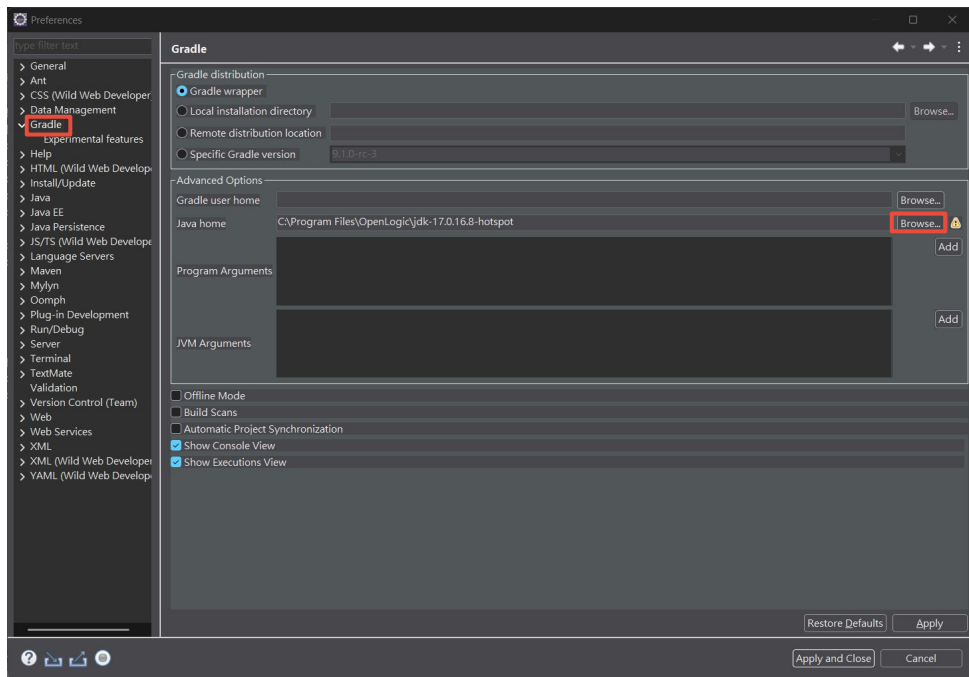
- ❖ 右鍵專案 → Properties → Gradle
- ❖ 找到 Advanced Options → Java home
- ❖ 點 Browse..., 選擇你安裝的 JDK 17 路徑, 例如:
 - ➡ C:\Users\<user>\AppData\Local\Programs\Eclipse Adoptium\jdk-17.0.16.8
- ❖ Apply 或 Apply and close
- ❖ 右鍵專案 → Gradle → Refresh Gradle Project



專案使用Gradle如何設定JDK?

配置路徑：

Window → Preferences → Gradle → Java home → Browse → 點選JDK位置 → Apply and Close



其他補充：為何已經設定了 Installed JREs還要另外設定 Gradle專案的Java

Installed JREs VS Gradle專案的Java home

什麼是 Installed JREs ?

Installed JREs是Eclipse IDE層面的Java運行環境配置, 用於管理 Eclipse可使用的所有 Java版本。

主要功能:

- **編譯Java程式碼:** Eclipse使用配置的JRE來編譯.java文件
- **語法檢查:** 根據選定的Java版本進行語法驗證
- **程式執行:** 在Eclipse內部運行Java應用程式
- **除錯功能:** 提供debug和profiling支援

作用範圍:

影響整個Eclipse工作區(Workspace)中的所有Java專案的編譯和執行環境。

Installed JREs VS Gradle專案的Java home

什麼是 Gradle Java home？

Gradle Java home是專門用於Gradle建置工具的Java環境配置，指定Gradle daemon和建置過程使用的JDK路徑。

主要功能：

- **執行Gradle任務：**運行gradle build, test, assemble等指令
- **依賴管理：**下載和管理專案依賴
- **建置生命週期：**控制專案的編譯、打包、測試流程
- **插件執行：**運行各種Gradle插件

作用範圍：

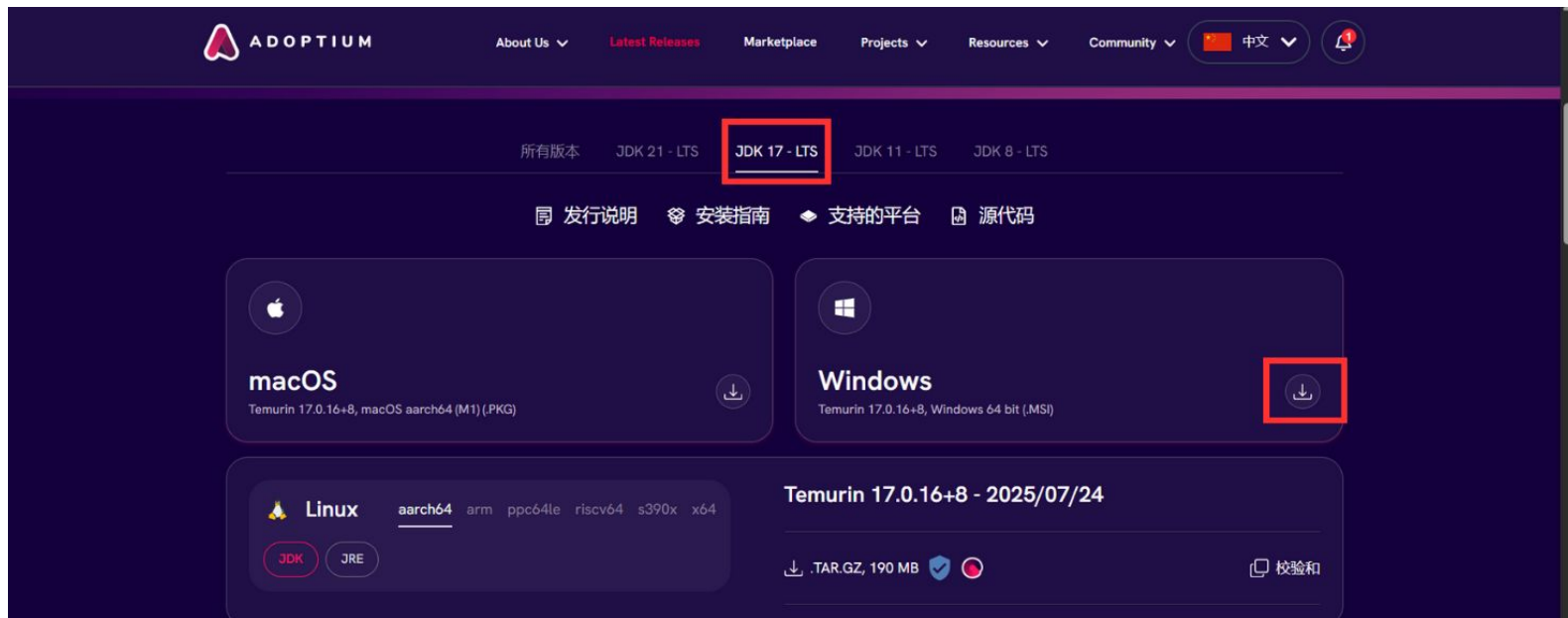
僅影響使用Gradle建置系統的專案，不影響一般的Java專案編譯。

詳細功能比較

比較項目	Installed JREs	Gradle Java home
主要用途	Eclipse IDE的Java編譯和執行環境	Gradle建置工具的執行環境
影響範圍	整個Eclipse工作區的所有Java專案	僅Gradle專案的建置過程
使用時機	編輯代碼時的語法檢查、在Eclipse內運行程式	執行gradle build、test等建置任務
可選版本	JRE或JDK都可以	必須是JDK (包含編譯器)
配置複雜度	相對簡單, 可管理多個版本	通常選擇一個主要版本

其他JDK檔下載安裝設定

第一步驟：下載安裝檔



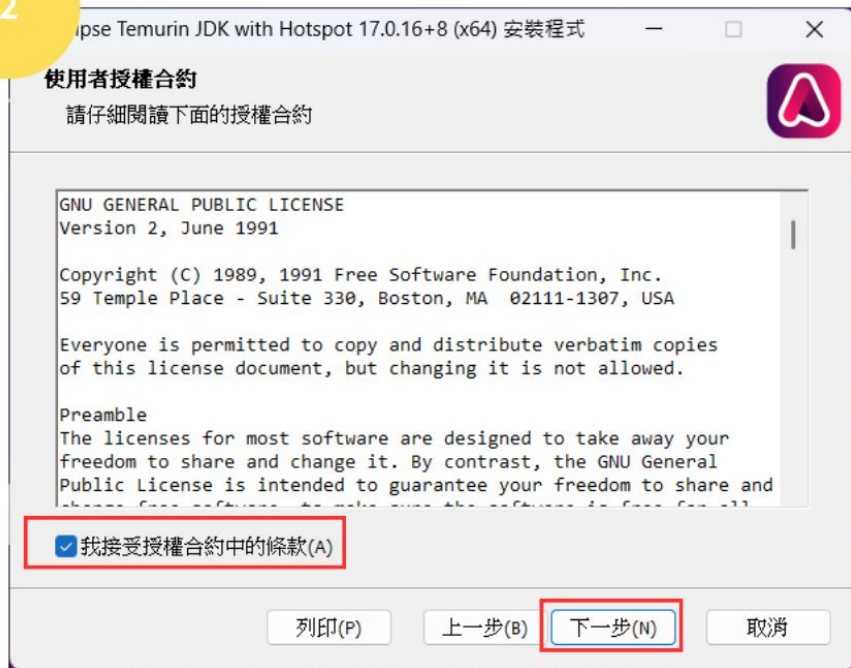
OpenJDK17U-jdk_x64_windows_hotspot_17.0.16_8.msi

第二步驟：執行安裝程式

1

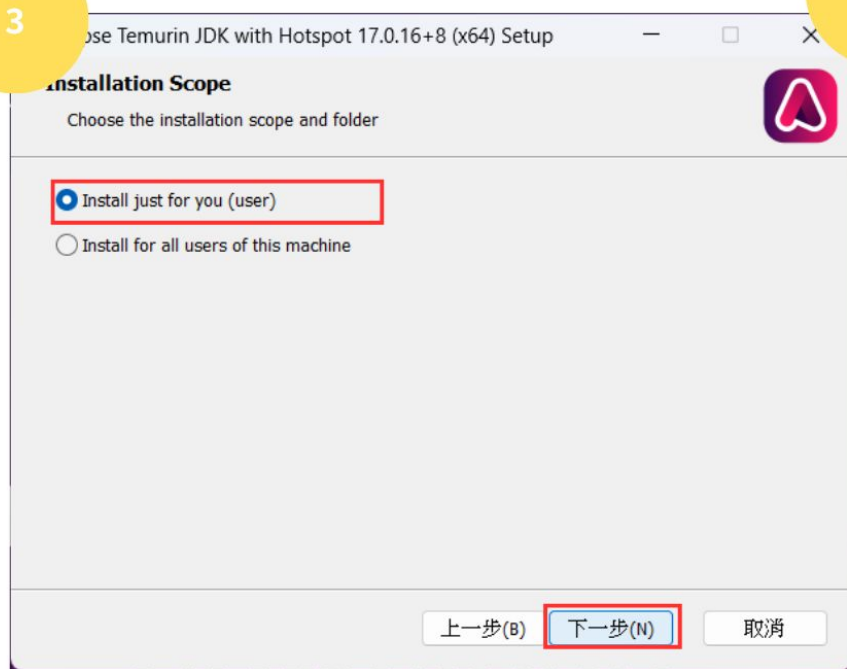


2

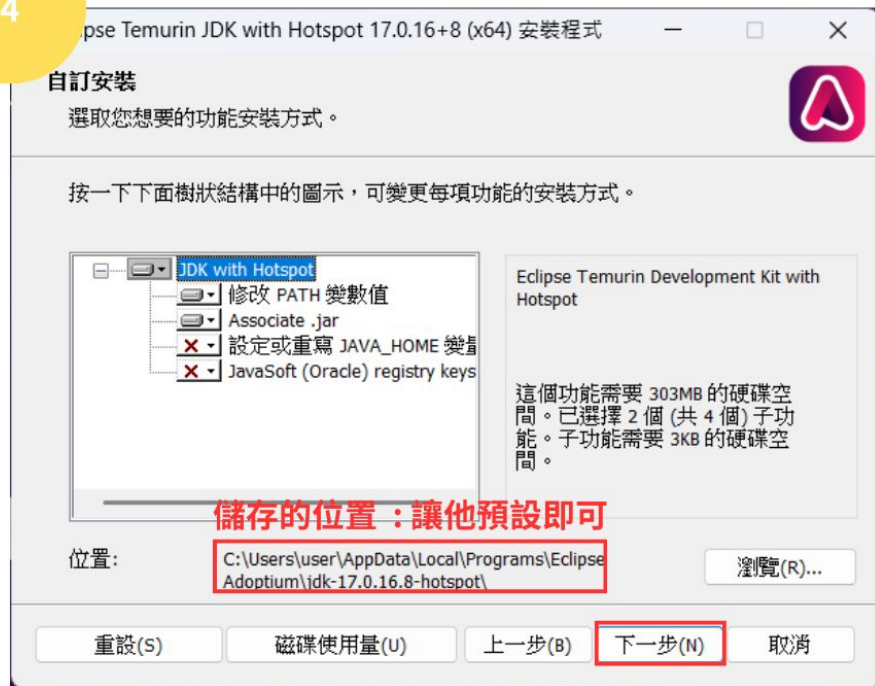


第二步驟：執行安裝程式

3



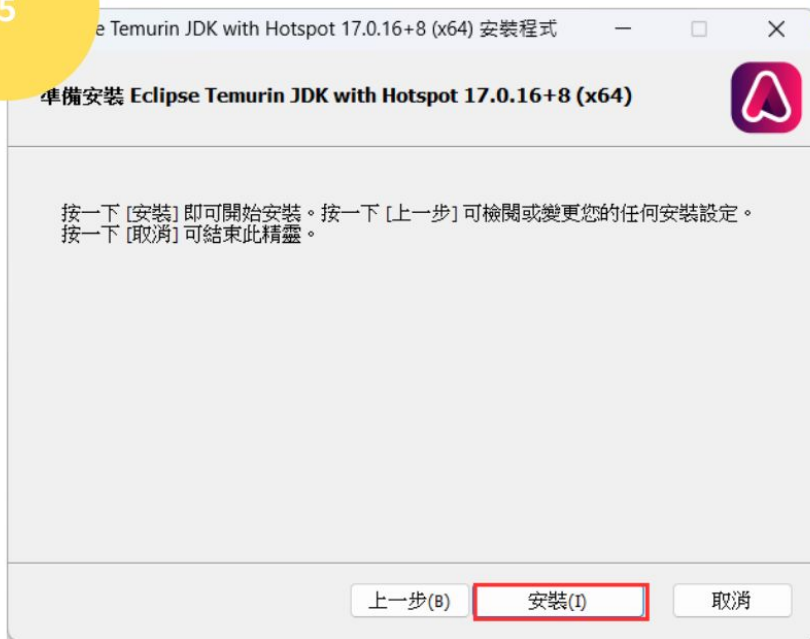
4



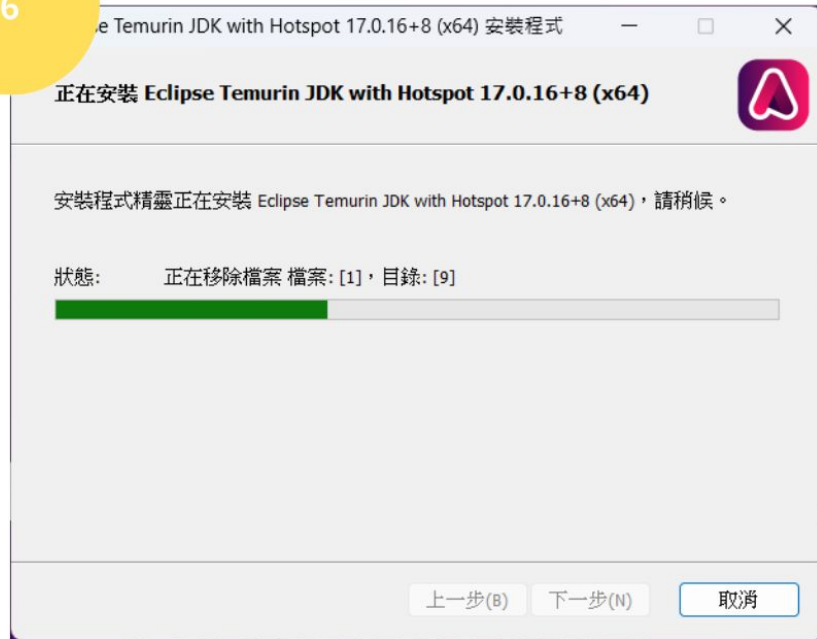
儲存的位置：讓他預設即可

第二步驟：執行安裝程式

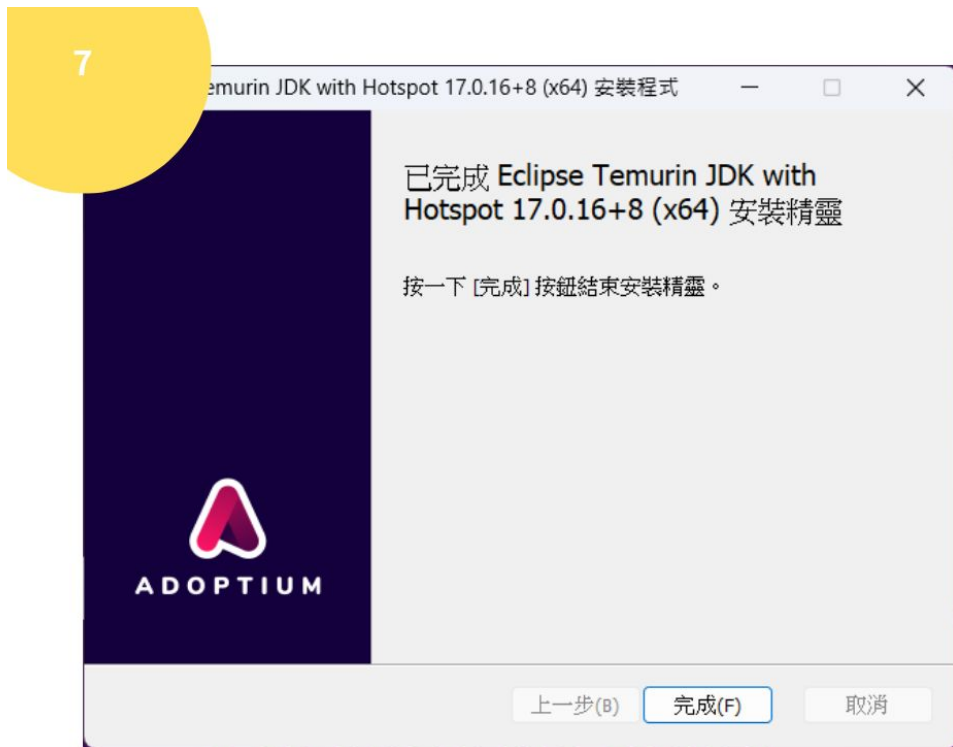
5



6



第二步驟：執行安裝程式



第三步驟：設定環境變數

1

設定 > 系統

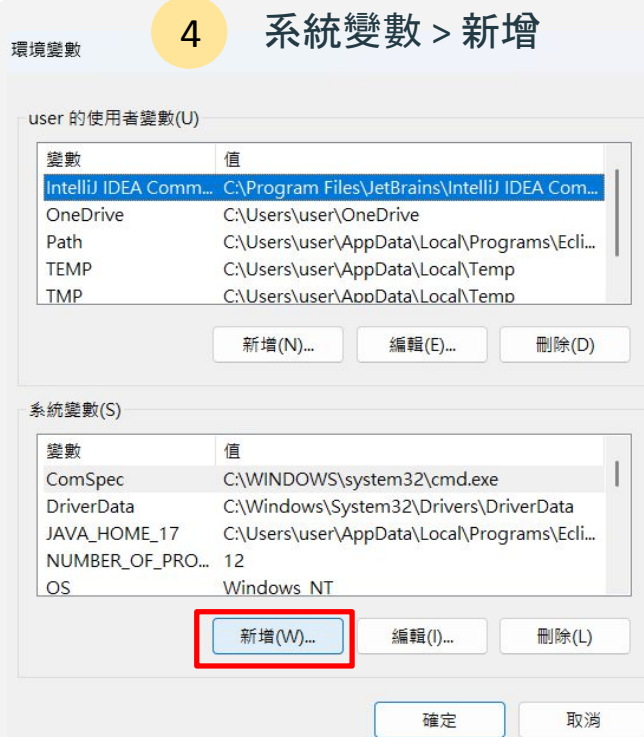


2

進階系統設定



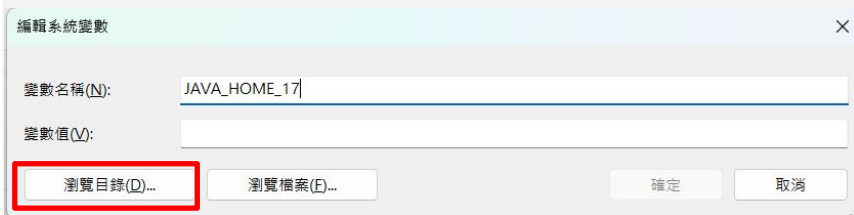
第三步驟：設定環境變數



第三步驟：設定環境變數

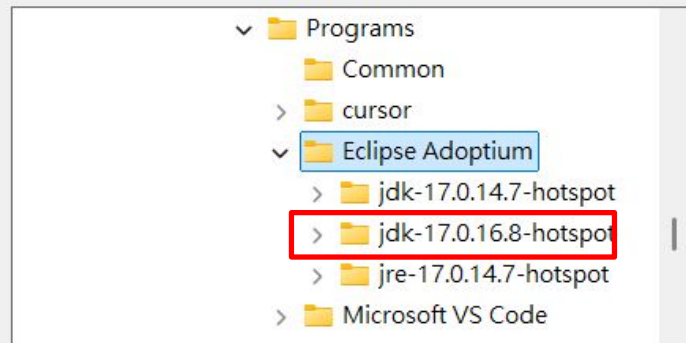
3 新增系統變數

- > 變數名稱: JAVA_HOME
- > 變數值: 請點選「瀏覽目錄」



4 尋找剛下載的JDK檔

瀏覽資料夾



資料夾(F): Eclipse Adoptium

建立新資料夾(M)

確定

取消

第三步驟：設定環境變數

5

點擊確定

新增系統變數

變數名稱(N): JAVA_HOME_17

變數值(V): C:\Users\user\AppData\Local\Programs\Eclipse Adoptium\jdk-17.0.16.8-hotspot

瀏覽目錄(D)...

瀏覽檔案(F)...

確定

取消

環境變數

6

在「系統變數」中

> 找到「Path」

user 的使用者變數(U)

變數	值
IntelliJ IDEA Comm...	C:\Program Files\JetBrains\IntelliJ IDEA Com...
OneDrive	C:\Users\user\OneDrive
Path	C:\Users\user\AppData\Local\Programs\Ecli...
TEMP	C:\Users\user\AppData\Local\Temp
TMP	C:\Users\user\AppData\Local\Temp

新增(N)...

編輯(E)...

刪除(D)

系統變數(S)

變數	值
NUMBER_OF_PROCES...	12
OS	Windows_NT
Path	C:\Windows\system32;C:\Windows;C:\Win...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF...

新增(W)...

編輯(I)...

刪除(L)

確定

取消

第三步驟：設定環境變數

5

點擊確定

新增系統變數

變數名稱(N): JAVA_HOME_17.0.16.8

變數值(V): C:\Users\user\AppData\Local\Programs\Eclipse Adoptium\jdk-17.0.16.8-hotspot

瀏覽目錄(D)...

瀏覽檔案(F)...

確定

取消

6

編輯Path

在「系統變數」中 > 找到「Path」> 編輯

user 的使用者變數(U)

變數	值
IntelliJ IDEA Comm...	C:\Program Files\JetBrains\IntelliJ IDEA Com...
OneDrive	C:\Users\user\OneDrive
Path	C:\Users\user\AppData\Local\Programs\Ecli...
TEMP	C:\Users\user\AppData\Local\Temp
TMP	C:\Users\user\AppData\Local\Temp

新增(N)...

編輯(E)...

刪除(D)

系統變數(S)

變數	值
NUMBER_OF_PROCES...	12
OS	Windows_NT
Path	C:\Windows\system32;C:\Windows;C:\Win...
PATHTEXT	;.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WS...

新增(W)...

編輯(I)...

刪除(L)

確定

取消

第三步驟：設定環境變數

7

新增

編輯環境變數

(1) 新增

新增(N)

編輯(E)

瀏覽(B)...

刪除(D)

上移(U)

下移(O)

編輯文字(T)...

C:\Windows\system32
C:\Windows
C:\Windows\System32\Wbem
C:\Windows\System32\WindowsPowerShell\v1.0\
C:\Windows\System32\OpenSSH\
C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR
%SystemRoot%\system32
%SystemRoot%
%SystemRoot%\System32\Wbem
%SYSTEMROOT%\System32\WindowsPowerShell\v1.0\
%SYSTEMROOT%\System32\OpenSSH\
C:\Program Files\nodejs\
C:\Program Files\Docker\Docker\resources\bin
C:\Program Files\Git\cmd
%JAVA_HOME_17%\bin

(2) 輸入「%JAVA_HOME_17%\bin」

(3)

確定

取消